OOMS

1) S.P.M / POP ⟶ C, B
2) OOM ⟶ C++, Java - -
3) TiM / P.P / C.M ⟶ Moni, Mupex - -
4) Logical Programming ⟶ Prolog ⟶ AI
5) functional programming ⟶ ML, LISP - - -

1972 unix operating sys. ⟶ 'c' lang
↓
most secured os

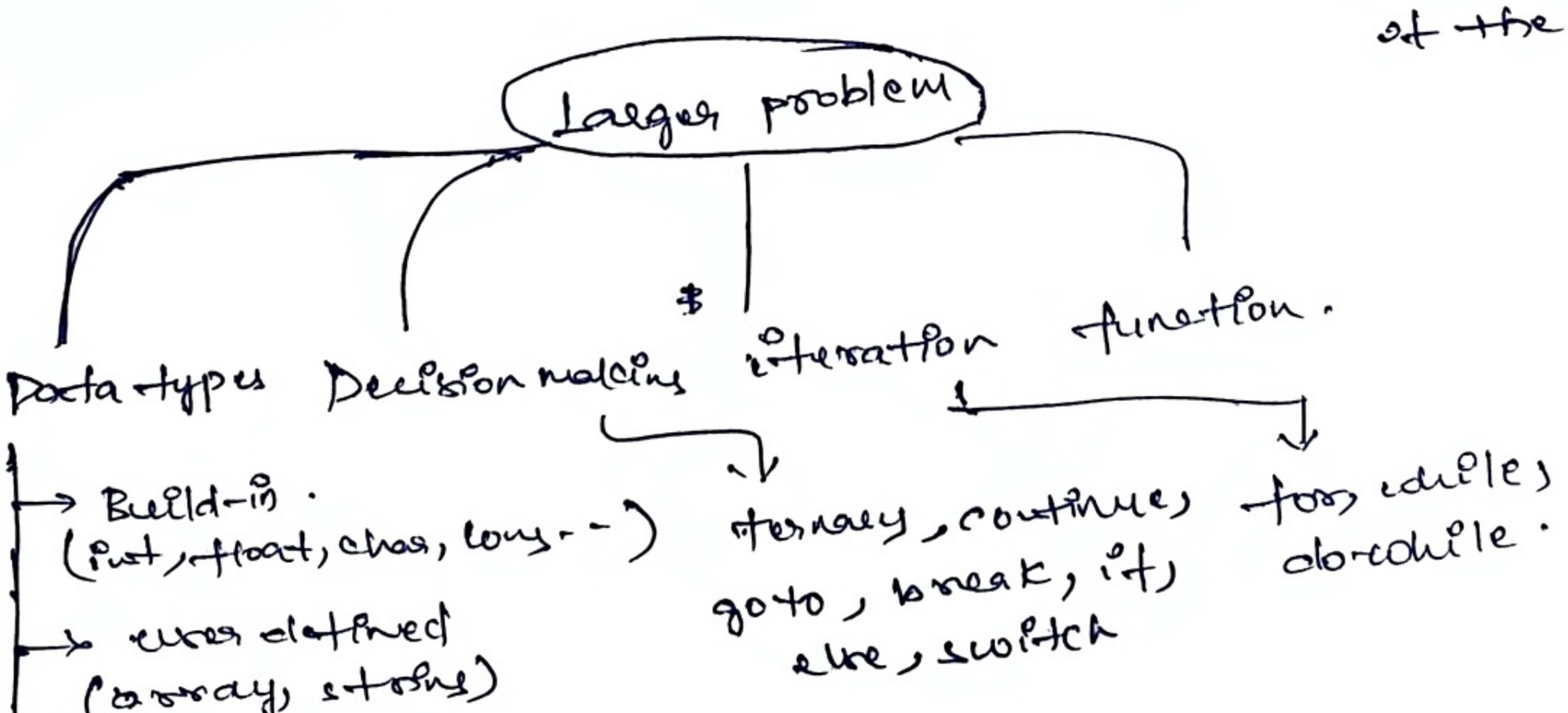
Structure oriented programming ⟶ top down approach
(sequence)

same operator used for various operations ⟵ • Abstraction } Less bound in S.O.P
• Reusability. } High bond in OOM

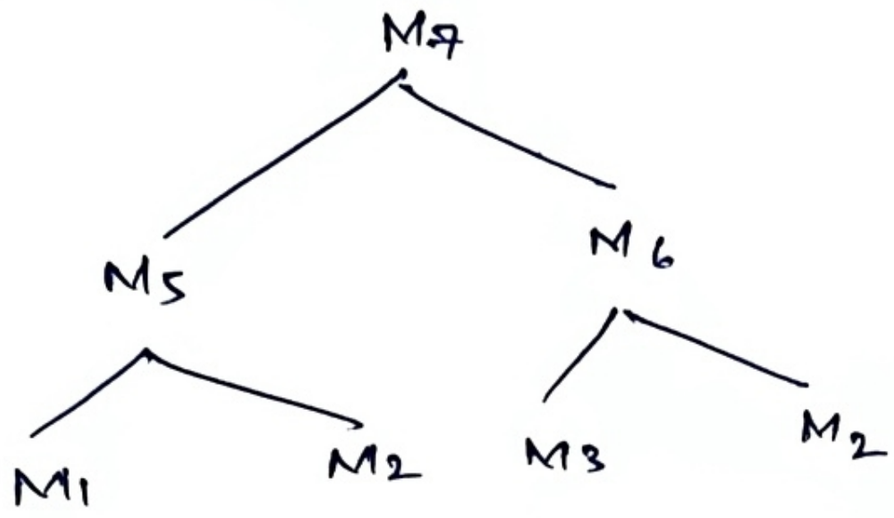
function: 4 component
→ fun name
→ " argument
→ " body
→ return

subroutine. ↓
3 component.
→ subroutine name
→ " argument
→ " body.

• reduces the overhead of the programme.

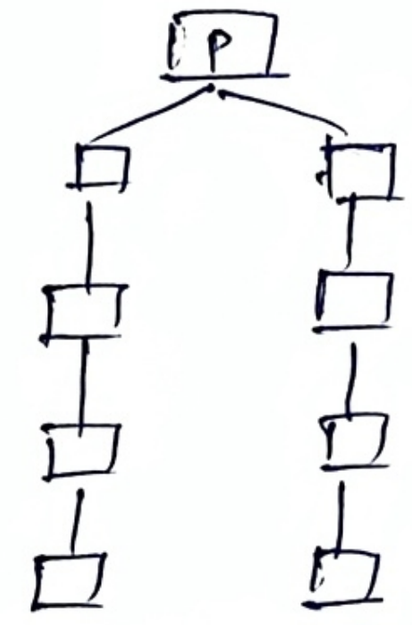


Bottom-up approach



→ we can work parallely

Top-down approach



→ we can't work parallelly.

function declaration → 3 comp → ;

function definition → H comp → int sum (int a, int b) {

int c;

c = a+b

}

int sum (int a, int b);

Error

↳ syntax
↳ semantic
↳ logic

a = 0
b = 2
c = b/a

```
int main (int argc, char * [arg v])
{
    int p, q, r;
    p = 2;
    q = 3;
    r = sum(p, q);
}
```

p, q values moves to the sum func & value returns from sum func to main func.

→ Not secure bcz data is moving

→ ~~Data~~

- reusability in OOPS is done by inheritance

Simulation → Artificial environment which ~~creates~~ ~~real~~ feels like original. with the help of programming lang.

c++ → 1) Robotics
2) Networking → creates ~~real~~ Artificial envi.
3) Simulation & emulation. using hardware & software.
4) CAD/CAM Lab → c++ / JAVA
to create ~~models of~~ (Mechanical models)
5) Automation systems

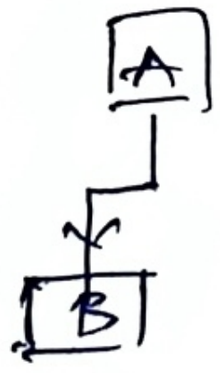
Features of OOP

1. Class / Object — O-1
2. Inheritance
3. Polymorphism
4. Abstraction
} O-3
5. Dynamic Binding
6. Encapsulation
} O-4
7. Add-on (Exception Handling) — O-5

→ Class — Blue print.

Class & Object are diff, but there in use often used together.

→ Inheritance



takes the properties.

many form

→ Polymorphism

• using same form multiple types.

• Simula → 1st

• Simula 67 → 2nd → 1967 } OOP

C → 1969-1970

C with classes (renamed)

C++

C++ → keyword. → class.

```
#include <iostream>          → operator
using namespace std;
```

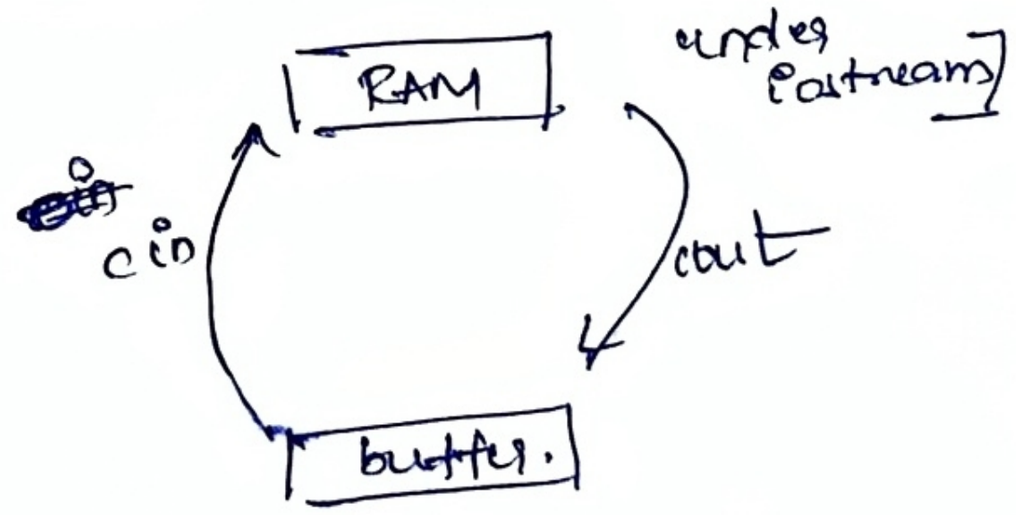
preprocessor

```
int main() {
    int a, b, c;
    cout << "Please enter two variables" << endl;
    cin >> a >> b;
    c = a * b;
    cout << "value of c" << c << endl;
    return 0;
}
```

argument, method

class → objects → cin, cout

[cin, cout → objects under iostream]



C

```
#include <stdio.h>
int main() {
    int a, b, c;
    printf("Please enter two variables \n");
    scanf("%d, %d", &a, &b);
    c = a * b;
    printf("Value of c is %d", c);
}
```

file → functions → printf, scanf

accessing object in class

↳ std::cout

std::cin

↳ scope resolution operator

↳ instead of writing like this,

we use . (using namespace std;)

↓

keyword

→ in C++

return type is compulsory, optional in C

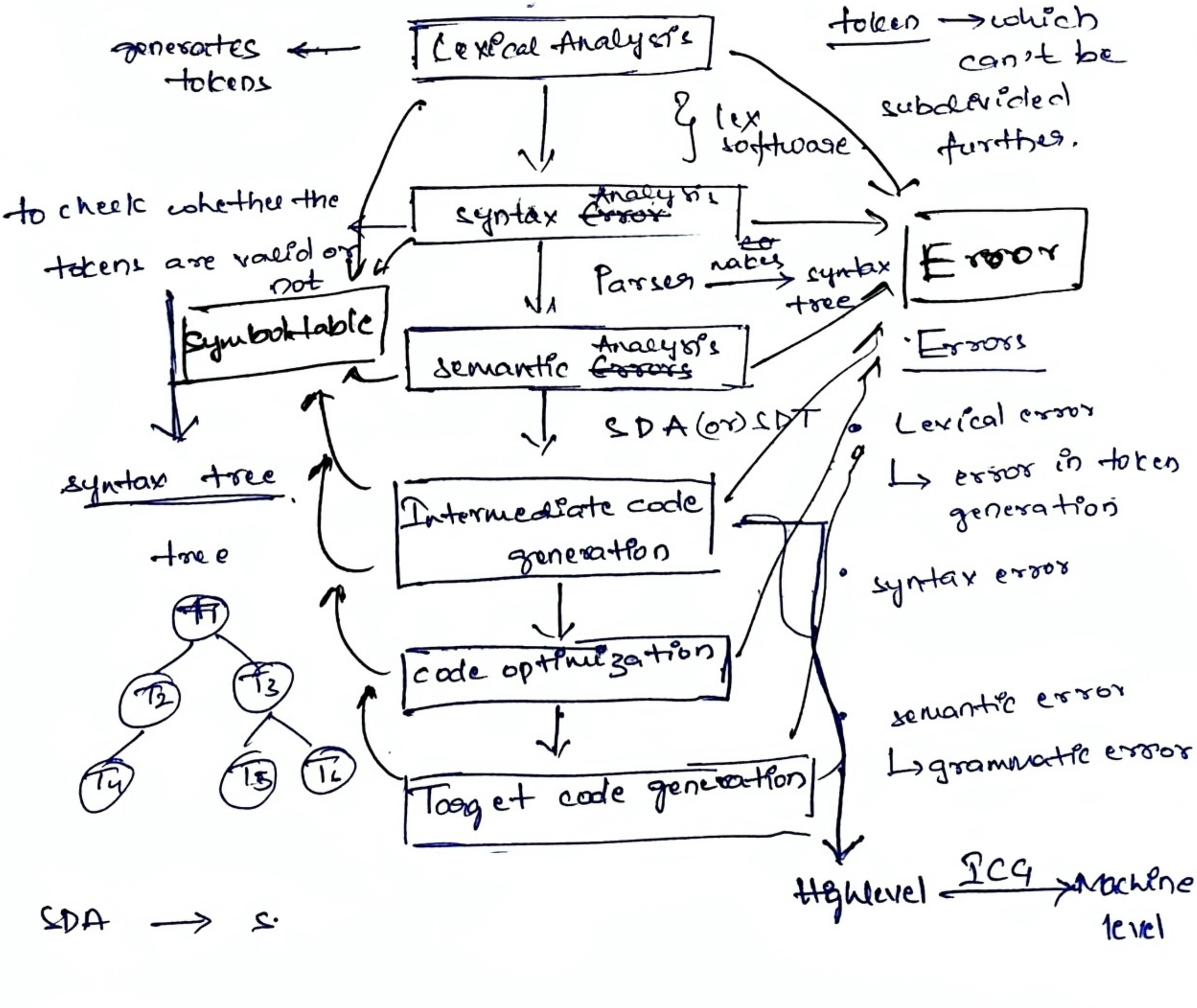
⟹ C++ compilation process

T1 T2 T3 T4 T5 T6

cout << "Hello World" >> endl ;

16 tokens.

before compilation ..linking & loading is done by linker & loader.



SDA → S.

Code optimization ⟶ lookes of code ⟶ less no. of lines.

↓

Target code generation (Binary lang)

Static

↳ global variable

text text

code ↳ code

stack

↳ local variable

Heap

↳ dynamic memory

C fdata types

C++ preprocessor

↳ files ⟶ #include

↳ Macros ⟶ #define.

↳ used for multiple occurance

#include <iostream>

#include "abc.txt"

- without using initialization & taking input from user.

#include "abc.txt"

(;) ⟶ semicolon is used for telling compiler to do tokenization

#define MAX 5

using namespace std;

int main() {

cout << MAX << endl;

return 0;

}

output

5

```
#include <iostream>
using namespace std;
#define PRINT_RECTANGLE (height, width)
        for (int i=0; i< width ; i++) {
            for (int j=0 ; j< height ; j++) {
                cout << "*";
            }
            cout << endl;
        }
int main() {
    PRINT_RETANGLE;
    return 0;
}
```

→ Memory segment (static.

---

cout (<<) → insertion operator → inserting value in buffer

cin (>>) → extraction operator. → extracting value from RAM

token — smallest individual unit.

keyword — reserved words with special meaning.

Identifies.

Datatypes

- User defined
  - class
  - structure
  - Union
  - Enum
- Build-In
  - Int — 4B
  - unsigned-Int — 4B
  - Signed-Int — 4B
  - short-Int — 2B
  - Long Int — 8B
  - float — 4B
  - Double — 8B
  - Boolean — 1B
  - char — 1B
- Derived.
  - Array
  - String
  - Pointer

~~#~~ include <iostream> OOMS

using namespace std;

~~struct~~ struct student {

char Name[100]; — 100 B

int age; — 4 B

struct DOB { — 12 B

int day; 116 B

int month;

int year;

} DOB_student;

};

int main() {

student s1;

s1.name = "Aman";

s1. age = 18;

s1. DOB_student. day = 5 ;

" . month = 6 ;

" . year = 7 ;

cout << sizeof (student) << endl.

}.

If we union instead of struct

⟶ size = 100 B

- in union

at a time only one memory can be stored.

. (Enumeration)

⟹ Enum is user defined datatype used to store integral constant.

Enum

enum e1 = { Jan=0, feb, Mar, Apr, May, June, July Aug, Sep, Oct, Nov, Dec };

int main() {

for (int = Jan; i<=Dec; i++) {

cout << i << endl; ⟶ output

1 2 3 --- 12

}

}

```
enum A= { a, b, c=4, d }
int main() {
    cout << a << endl.      ⟶ 0
    "    b    "            ⟶ 1
    "    c    "            ⟶ 4
    "    d    "            ⟶ 8
```

Q)

```
enum colors { red=5, black };
enum suit { heart, diamond=8, spade=3, club };
int main() {
  cout << red << black << endl;  ⟶  5 . 6
  cout << heart << diamond << spade << club <<
```

0 8 3 4

Q) ~~enum cat {~~

~~temp=7, tem.~~

» diff btw char arr and string is null character

→ logical and operator

→ arithmet · a & b

a * b

→ referencing operator

int *p

→ address operator

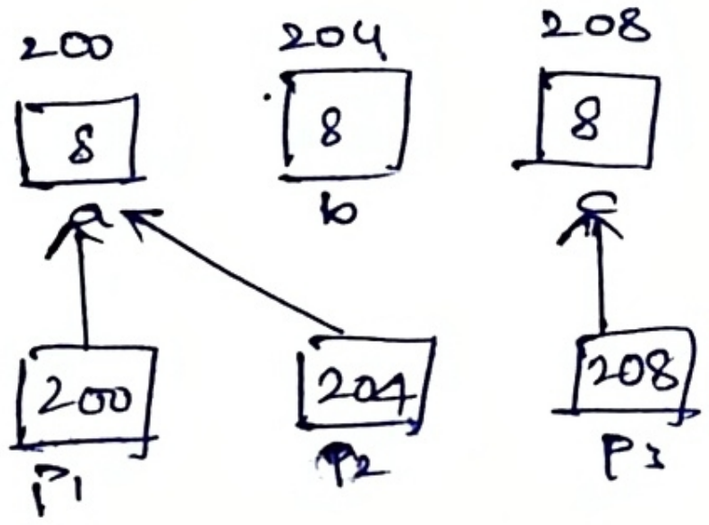
p = &a

Pointers

Ex

```
# include <iostream>
using namespace std;
int main() {
int a=100 , b=88 , c=8;
int *p1 = &a , *p2 , *p3 = &c ;
   p2 = &b;
   p2 = .p1;
   b = * p3;
  *p2 = * p3;
cout << a << b << c;
        8   8   8
return 0;
}
```

int *p2 = Null
↳ doesn't store garbage value



⟹

```
int main() {
 int a = 120 , b = 80, c=40
   int * p1 = &a , * p2 = &b , * p3 = &c
   * p1 = * p2 + * p3
   p2 = p1;
  * p2. = * p3 + 10;
   p3 = &b;
  * p1 = * p2 + * p3;
   b = * p1 - * p3;
  * p3 = * p1 - b;
   cout << a << b << c;
           120  80  40
   return 0;
  }
```

⟹ int main (){

int a = 50 , b=30 , c=20;

int * p1=&a , *p2= &b , *p3 =&c;

*p1 = *p2 + *p3;

p3 = p1;

*p2 = *p3 - 10;

*p1 = *p2 + *p3;

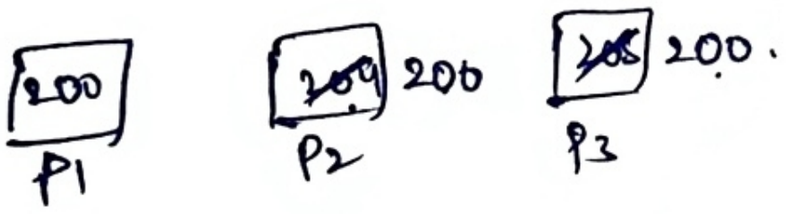
p3 = p1;

*p2 = *p1 - a;

cout <<a<<b<<c;

0 40 20





⟹

int a=10;

int *p= NULL

if(p) → F

if(!p) → T

if (p) {

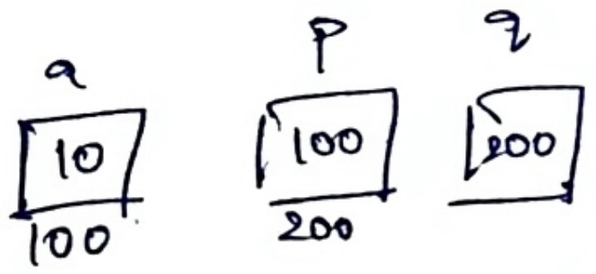
} else {

→ Pointer to Pointer

⟹ . int a =10;

int *p = &a;

int **q = &p;



q= &a;

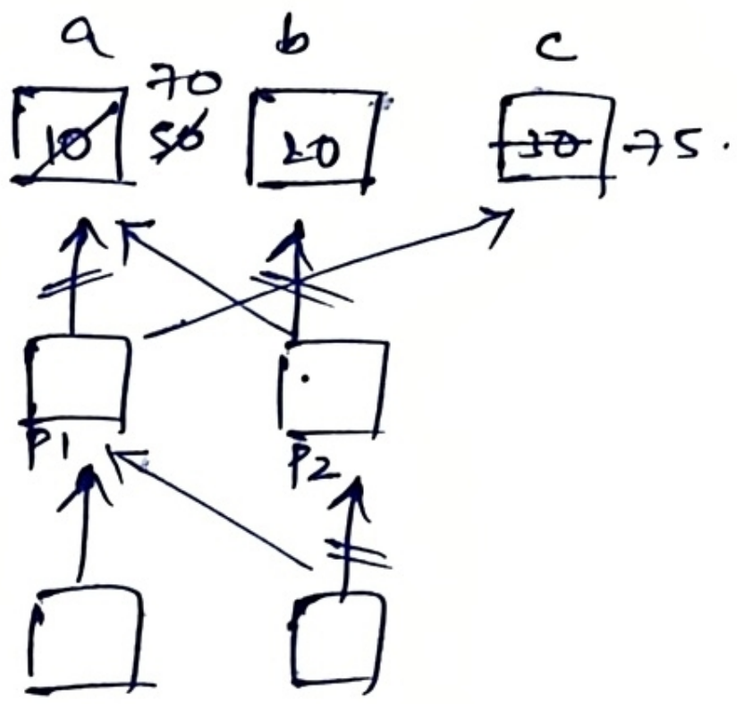
cout << a → 10

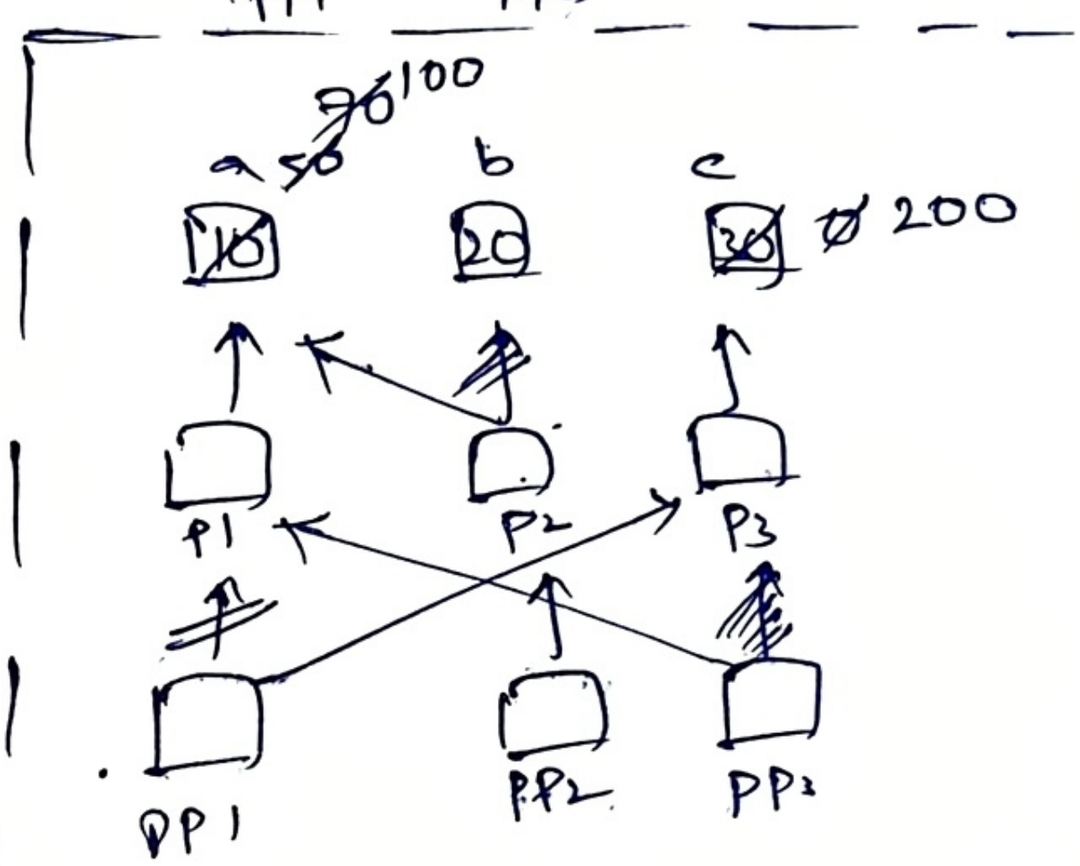
cout << *p → 10

cout << **q → 10

⟹

```
#include <iostream>
using namespace std;
int main() {
    int a = 10, b = 20, c = 30;
    int *p1 = &a;
    int *p2 = &b;
    int **pp1 = &p1;
    int **pp2 = &p2;
    **pp1 = **pp2 + c;
    p2 = p1;
    pp2 = &p1;
    **pp2 = a + b;
    p1 = &c;
    **pp1 = *p2 + 5;
    cout << a << b << c;
    return 0;
}
```



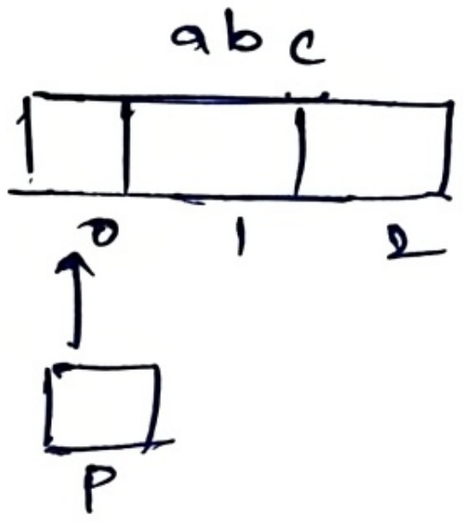


⟹

```
#include <iostream>
using namespace std;
int main() {
    int a=10, b=20, c=30;
    int *p1 = &a;
    int *p2 = &b;
    int *p3 = &c;
    int **pp1 = &p1;
    int **pp2 = &p2;
    int **pp3 = &p3;
    **pp1 = **pp2 + **pp3;
    **pp2 = *p1 - *p2;
    pp3 = pp1;
    **pp3 = **pp1 + *p2;
    p2 = p1
    **pp2 = *p2 * + *p1;
    pp1 = &p2;
    **pp1 = *p2 - *p1;
    pp1 = &b; → compile time error
    **pp1 = *p1 + *p2
    cout << a << b << c;
    return 0;
}
```

a = 100
b = 20
c = 200

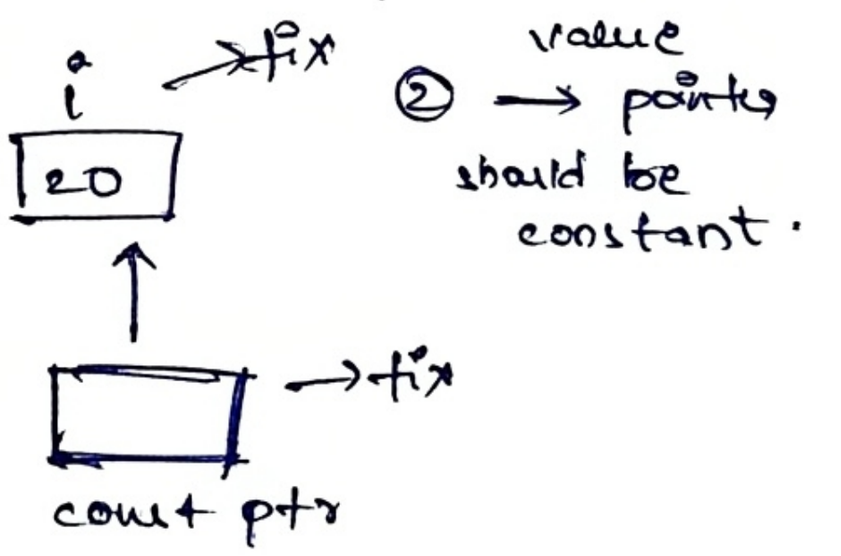
```
int main(){
    int abc[3];
    int *p;
    p = abc;
```




```
int main (){
    double abc[3]; → 24B
    double *p[3]; → 12B  (each every time 4B)
      p[0] = abc;
     p[1] = abc+1;
     p[2] = abc+2;
}
```

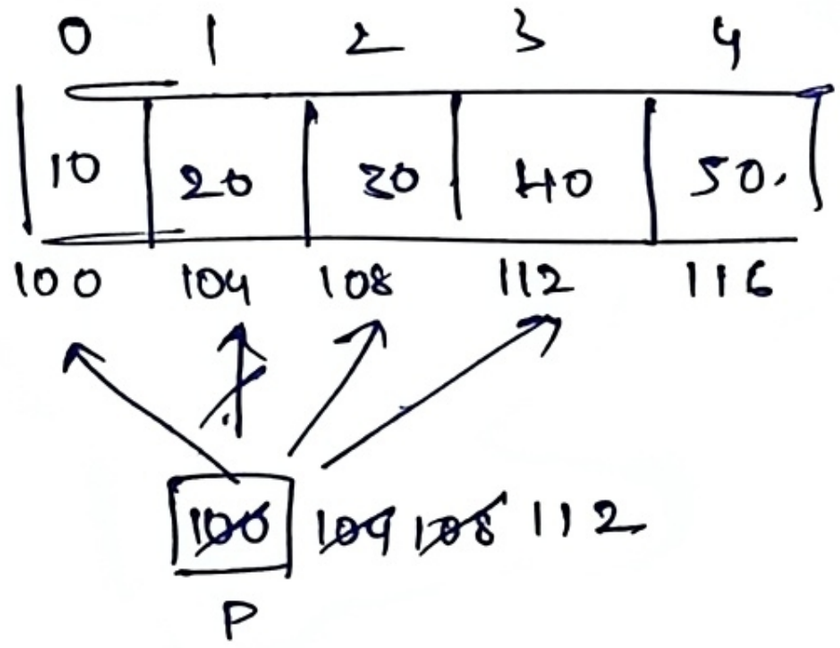
we should mention two
cout. ① → for pointing const
value
② → pointer should be constant.

```
#include <iostream>
using namespace std;
int main(){
    const int i = 20;
    const int * const ptr = &i;
    (* ptr)++;
    int j = 15;
    ptr = &j ⟶ compile time error
    cout << i;
    return 0;
}
```



→

```
#include <iostream>
using namespace std;
int main(){
    int num[5];
    int *p;
    p = num;
    *p = 10;
    p++;
    *p = 20;
    p = &num[2];
    *p = 30;
    p = num + 3;
    *p = 40;
    *(p+1) = 50;
    for (int i=0; i<5; i++) {
        cout << num[i] << endl;
    }
    return 0;
}
```



output

```
10
20
30
40
50
```

→

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {5,10,15,20,25,30,35};
    int *p = arr+1;
    int *q = arr+5;
    int x = (*p++) + (*--q);
    int y = *(p+2) - *(q-1);
}
```

*p++, *--q → value doesn't change (address)

(*p)++ + *--q → value + address → compile time error

p = 10
q = 20

x → 40

y → 5

p = p+1; → 15

q = q-2; → 20

p = 15
q = 20

int z = *p + *(q+1); → 40

cout << x << y << z << endl;

cout << *(arr + (*p/5)) + *(q-1); → 35

return 0;

}

output 40 -5 40 35 //

→

```
int main(){
int arr[] = {4,5,6,7}
    int *p = (arr+1); → 5
     cout << *arr+10; → 14
      return 0;
     }
```

→

```
int main(){
int arr[] = {4,5,6,7}
    int &p = arr[0];
     cout << p; → 4
     cout << sizeof(p); → it shows memory
```

reference operator store address

(4) what it points

int char arr[] = {a,b,c,d}

cout << sizeof(p) → 1

→

string hello = "hello"

↓ ↓ ↓

class object value

string &hello_ref = hello

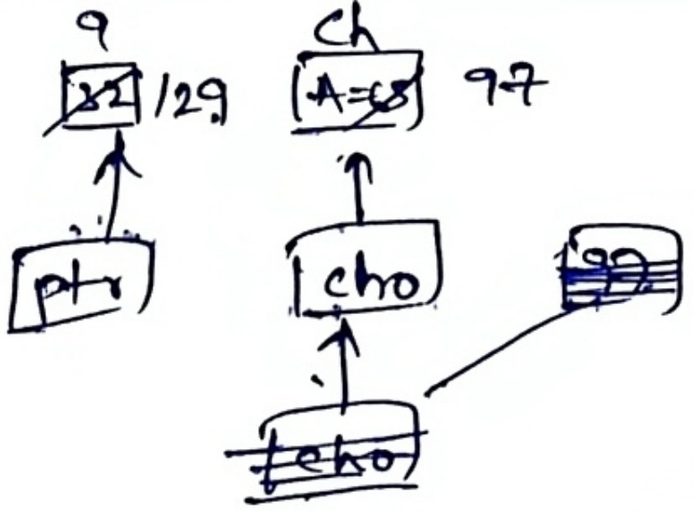
cout << hello_ref

↳ hello.

string *hello_ptr = hello;



```
#include <iostream>
using namespace std;
int main() {
  int a = 32 , *ptr = &a;
  char ch='A';
  char &cho=ch;
  cho+ =a;
 *ptr+ =ch;
 cout<<a<<cho<<endl;
 return 0;
}
```



~~ch=A~~

~~cho=A A~~

~~cho+=32~~

cho = 65

cho+ = a

↳ 65+32 = 97

*ptr = *ptr + ch

= 32 + (65)

= ~~32~~ 129

→

Operators in C++

① ⟶ input/output · "<<" ">>"

② ⟶ Arithmatic (+, -, *, /, %)

③ ⟶ Relational (<, ==, >, <=, >=, !=)

④ ⟶ logical (&&, ||, !)

⑤ ⟶ Bitwise (&, |, <, >)

» Scope Resolution operator (::)

```
int a = 10;
int main() {
    int a=20;
    cout<<::a;
```

↳ ↳ 10 (print the out of the scope value)

» This operator (⟶)

→ is combination of * .

used in structure

3) >> New / Delete

↳ Allocates memory

↳ deletes memory

```
int main() {
    int n;
    cin >> n;
    int arr[
```